

Intro to Deep Learning

Lecture 06: Deeper CNN Training

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. Why is CIFAR-10 harder than FashionMNIST?
2. What goes wrong when CNNs become deeper?
3. What does BatchNorm do?
4. What changes between BatchNorm train and eval mode?
5. Why do image augmentations help?
6. What is the main idea behind ResNet?

Lecture rhythm: deeper CNNs -> BatchNorm -> augmentation -> residuals

Recap From Lecture 05

A basic CNN classifier:

```
image  
-> convolution blocks  
-> flatten or pooling  
-> linear classifier  
-> logits
```

Common block:

```
Conv2d -> ReLU -> MaxPool2d
```

Today we ask: how do we train stronger CNNs?

Part 1

From FashionMNIST To CIFAR-10

Similar tensor size, harder visual problem

Dataset Shift In Difficulty

CIFAR-10 is only slightly bigger, but visually harder

FashionMNIST

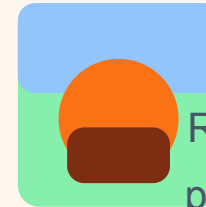


[B, 1, 28, 28]

grayscale

centered objects

CIFAR-10



[B, 3, 32, 32]

RGB + backgrounds
pose, lighting, clutter

The tensor shape is only slightly larger. The visual problem is much harder.

Why CIFAR-10 Is Harder

CIFAR-10 contains:

- color
- backgrounds
- object pose variation
- lighting variation
- more visual clutter
- objects at different positions

FashionMNIST is cleaner and more centered.

CIFAR-10 makes training tricks more visible.

Baseline CNN For CIFAR-10

Seminar baseline:

```
Conv(3 -> 32) -> ReLU -> Pool  
Conv(32 -> 64) -> ReLU -> Pool  
Flatten -> Linear -> ReLU -> Linear(10)
```

Important change:

```
in_channels = 3
```

CIFAR-10 images are RGB.

Deeper CNNs Are Useful

More layers can learn richer visual features.

Early layers:

edges, colors, simple textures

Later layers:

parts, shapes, class-relevant patterns

But deeper networks are harder to optimize.

What Can Go Wrong?

When CNNs become deeper, we may see:

- unstable loss curves
- slow convergence
- sensitivity to initialization
- sensitivity to learning rate
- train accuracy not improving as expected

This does not always mean the task is impossible.

It can mean the training dynamics are poor.

Part 2

Batch Normalization

Stabilize intermediate activations

Activation Distributions Shift

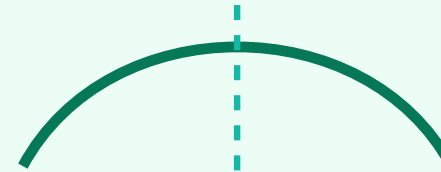
Deeper layers see moving activation distributions

without BatchNorm



distribution shifts as earlier layers change

with BatchNorm



normalize, then learn scale and shift

BatchNorm helps make intermediate values better behaved.

BatchNorm: Main Idea

Normalize intermediate activations inside the network.

For a mini-batch:

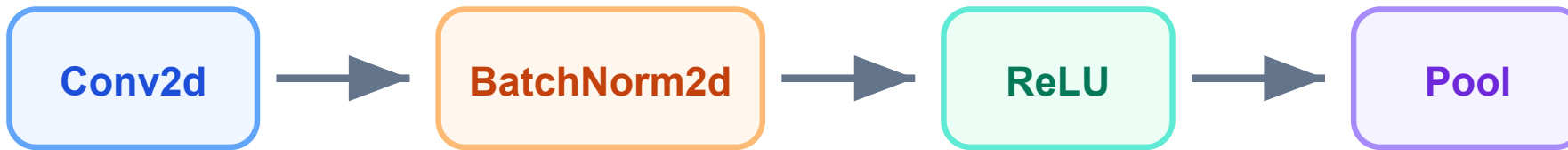
```
subtract batch mean  
divide by batch standard deviation
```

Then learn a new scale and shift.

This often makes training faster and more stable.

BatchNorm In A CNN Block

BatchNorm sits inside the convolution block



BatchNorm changes activation statistics, not tensor shape.

In seminar, the practical pattern is:

Conv -> BatchNorm -> ReLU -> Pool

BatchNorm Formula

For activations in a mini-batch:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Then:

$$y = \gamma \hat{x} + \beta$$

where:

```
gamma = learned scale  
beta  = learned shift
```

Why Scale And Shift?

If BatchNorm only normalized, it might remove useful information.

So the layer learns:

```
gamma: how much scale to restore  
beta:  how much shift to restore
```

This gives the model flexibility.

The network can use normalized activations without being forced into one fixed distribution.

BatchNorm In CNNs

For CNNs, use:

```
nn.BatchNorm2d(num_channels)
```

It normalizes each channel.

Example:

```
input shape: [B, 32, H, W]  
BatchNorm2d(32)  
output shape: [B, 32, H, W]
```

BatchNorm does not change tensor shape.

Where To Put BatchNorm

Common pattern:

```
Conv2d -> BatchNorm2d -> ReLU
```

The `num_features` of BatchNorm should match:

```
number of convolution output channels
```

For `Conv2d(3, 32, ...)`, use:

```
nn.BatchNorm2d(32)
```

Shape Example

Layer:

```
nn.Conv2d(3, 32, kernel_size=3, padding=1)  
nn.BatchNorm2d(32)  
nn.ReLU()
```

Shape:

```
[B, 3, 32, 32]  
-> [B, 32, 32, 32]  
-> [B, 32, 32, 32]  
-> [B, 32, 32, 32]
```

Part 3

Train Mode vs Eval Mode

BatchNorm is stateful

BatchNorm Train vs Eval

BatchNorm uses different statistics in `train()` and `eval()`

model.train()

current
batch stats

normalize with
batch mean / variance
update running stats

model.eval()

stored
running stats

normalize predictably
no stat updates

This is why mode switching matters.

Running Statistics

During training, BatchNorm tracks:

```
running mean  
running variance
```

These are not learned by gradient descent.

They are updated from batches during training.

At evaluation time, we cannot depend on the exact validation batch composition.

So we use the running statistics.

BatchNorm And Batch Size

BatchNorm estimates statistics from the mini-batch.

If batch size is very small:

```
batch mean and variance are noisy
```

Possible symptoms:

- unstable validation accuracy
- train/eval mismatch
- sensitivity to batch composition

For this course, remember the practical warning.

BatchNorm Is Not Magic

BatchNorm often helps, but it does not replace:

- enough data
- a reasonable learning rate
- correct preprocessing
- regularization when needed
- debugging tensor shapes

It is a training aid, not a guarantee.

In Seminar 06, you will compare it experimentally.

Part 4

Image Augmentation

Improve generalization by preserving labels under variation

Data Augmentation

Data augmentation creates modified training examples.

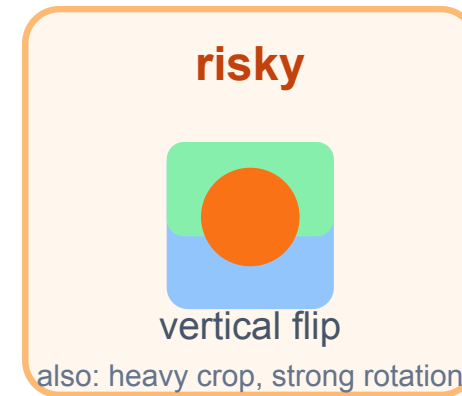
For images, examples include:

- horizontal flips
- random crops
- small rotations
- color changes
- blur or noise

The label should stay the same after the transform.

Safe vs Unsafe Augmentations

Augmentation should preserve the class



Always think about the dataset.

Why Augmentation Helps

Augmentation teaches the model:

```
small visual changes should not change the class
```

It increases effective dataset diversity.

It can reduce overfitting by making memorization harder.

For CIFAR-10, augmentation is often very important.

Train Only

Apply random augmentation to the training set.

Do not apply random augmentation to validation or test sets.

Validation should answer:

how does the model perform on clean held-out data?

If validation changes randomly every epoch, comparison becomes noisy.

Albumentations

Albumentations is a popular image augmentation library.

In Seminar 06, you will use it for:

```
HorizontalFlip  
RandomCrop with padding  
Normalize
```

Typical pipeline:

```
image -> augmentation -> tensor -> model
```

The output must match PyTorch image shape and dtype.

Normalization Is Also Preprocessing

Image normalization usually means:

```
x_normalized = (x - mean) / std
```

For RGB images:

```
mean and std are per channel
```

Normalization makes input scale more convenient for optimization.

It is not the same as BatchNorm.

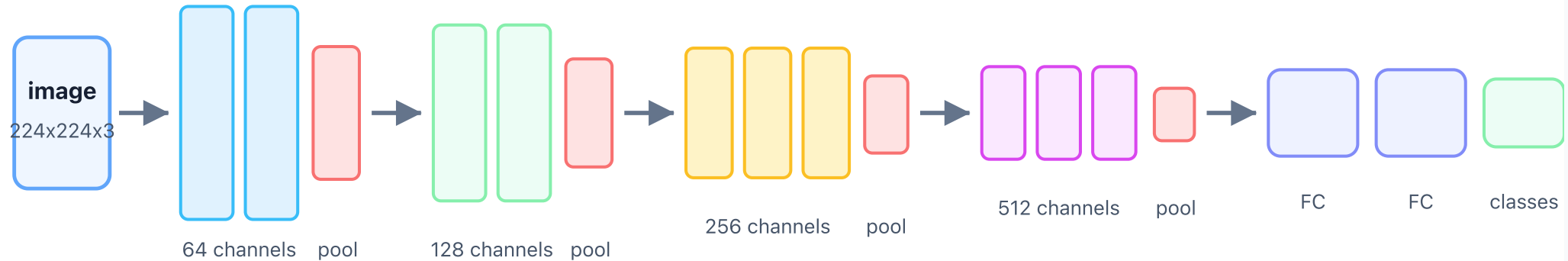
Part 5

Residual Connections

Make deeper networks easier to optimize

Before ResNet: Plain Deep CNNs

Plain deep CNN idea: stack many convolution layers



VGG-style networks were powerful: simple repeated blocks, more depth, more channels, smaller spatial maps.

VGG-style networks showed that simple repeated blocks can work very well.

The natural question:

can we just keep adding more convolution layers?

Why Plain Depth Becomes Hard

What becomes hard in very deep plain CNNs?



Even if extra layers are unnecessary, a plain stack has to learn an identity mapping through weights.
That is possible in theory, but unnecessarily hard during optimization.

The issue is often **optimization**, not expressive power.

Gradient Through Many Layers

For a plain deep network:

```
x -> f1 -> f2 -> f3 -> ... -> fL -> loss
```

the backward signal is a product of many derivatives:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial h_L} \frac{\partial h_L}{\partial h_{L-1}} \dots \frac{\partial h_1}{\partial x}$$

If many factors shrink the signal, early layers learn slowly.

If many factors amplify it, training can become unstable.

Residual Connection

Residual connections add the input back to a transformed version.

Instead of forcing a block to learn the whole mapping:

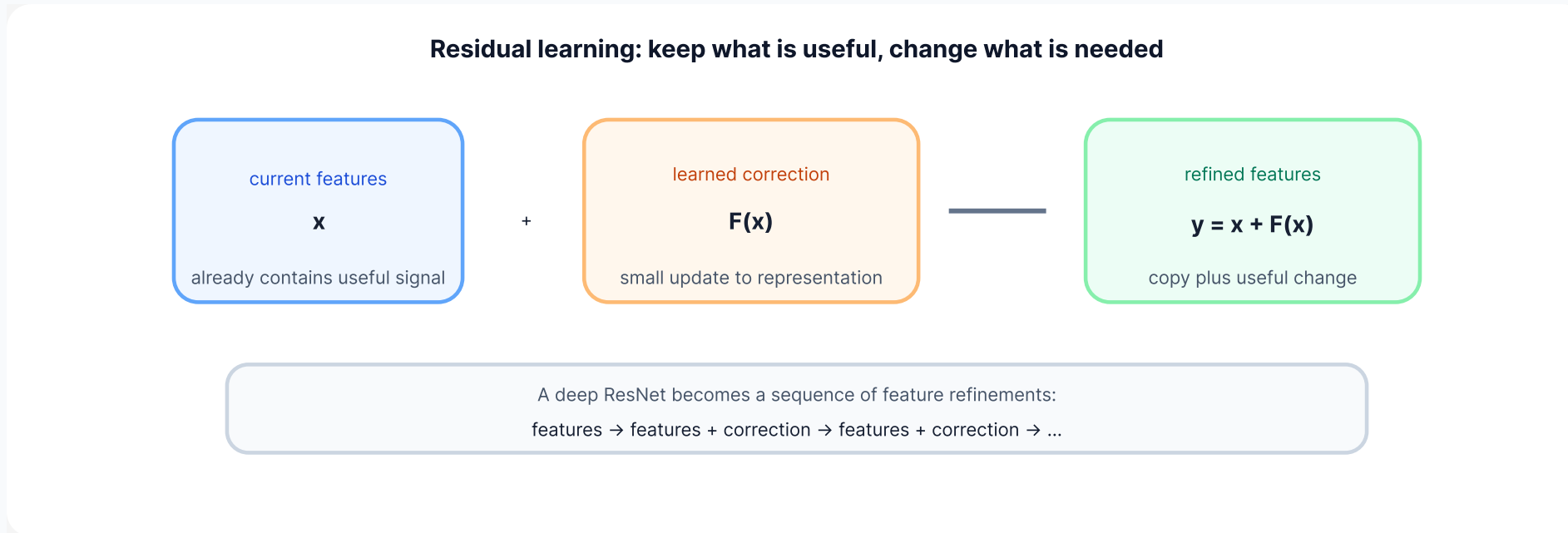
$$y = F(x)$$

use:

$$y = F(x) + x$$

This is the key idea behind ResNet.

Residual Block As A Correction



Think of the block as:

keep current features + learn a useful correction

If no change is needed, the block can learn $F(x)$ near zero, so y stays close to x .

Gradient Flow Intuition

For a residual block:

$$y = F(x) + x$$

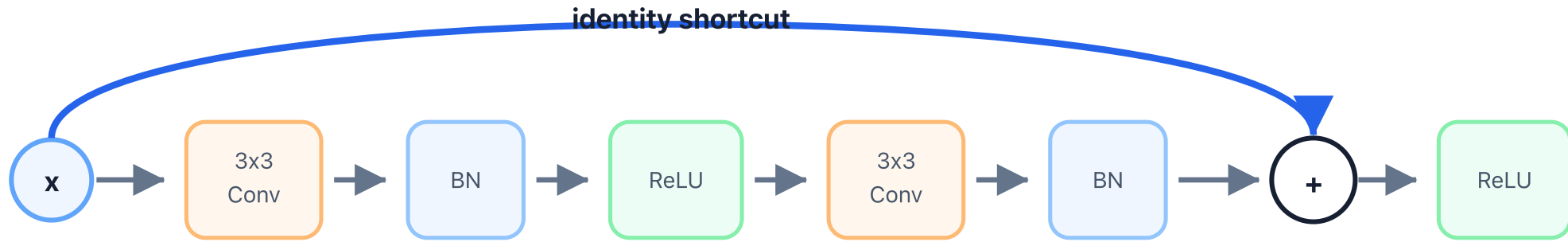
The gradient has two paths:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(\frac{\partial F}{\partial x} + I \right)$$

The identity term gives a more direct route for gradient signal.

Detailed ResNet Block

Basic ResNet block: transform branch plus skip branch

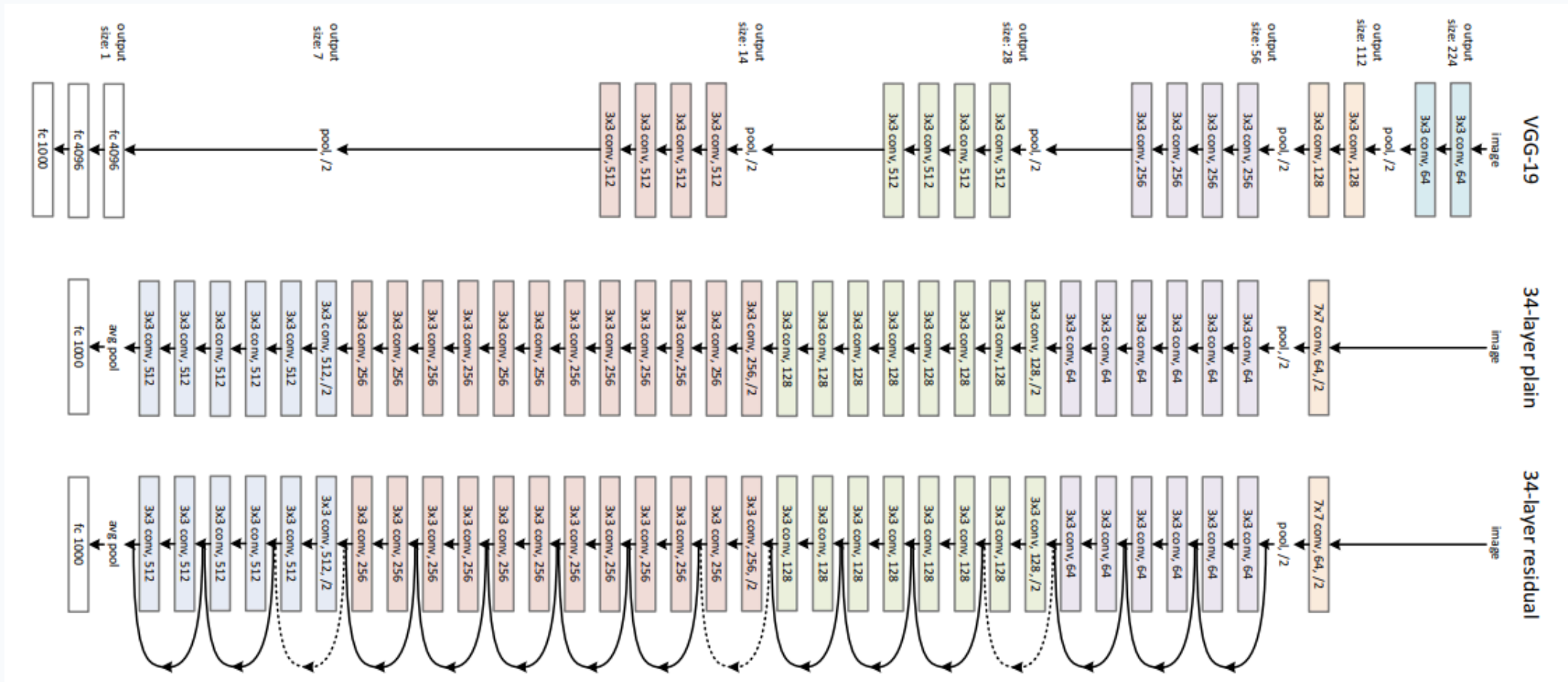


If spatial size or channel count changes, the skip path uses a projection, often a 1x1 convolution.
If shapes match, the shortcut can be the identity.

For the simplest residual block, input and output shapes must match.

If shapes differ, real ResNets use projection shortcuts.

ResNet Architecture



A full ResNet repeats residual blocks at several spatial scales.

Residuals Are Not Only For CNNs

Residual connections appear in many modern architectures:

- ResNets
- transformers
- diffusion models
- large language models

The common goal:

make deep computation easier to train

We will see residuals again in transformers.

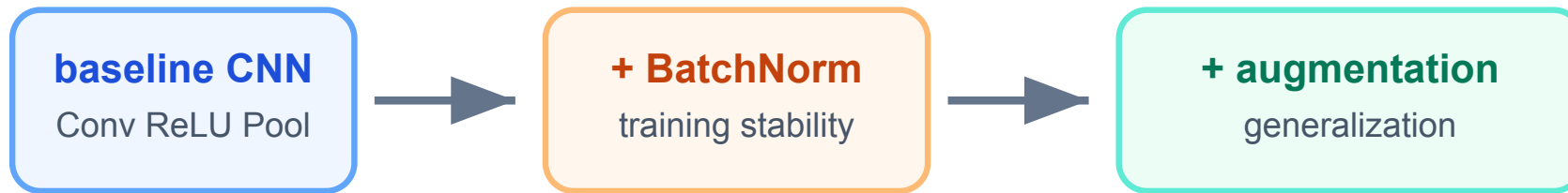
Part 6

Practical Recipe

Compare changes experimentally

Augmentation And BatchNorm Together

Compare one training improvement at a time



Track train loss, validation loss, validation accuracy, transforms, and notes.

Expected pattern:

BatchNorm improves training stability
augmentation improves generalization

Practical CNN Recipe

For a stronger small CNN:

```
Conv -> BatchNorm -> ReLU -> Pool  
Conv -> BatchNorm -> ReLU -> Pool  
Classifier
```

Training setup:

```
Adam or SGD with momentum  
CrossEntropyLoss  
train augmentation  
clean validation  
experiment logging
```

Common Bugs In Seminar 06

Bug 1:

```
using Conv2d(1, ...) for CIFAR-10
```

Bug 2:

```
Albumentations returns wrong dtype or shape
```

Bug 3:

```
augmenting validation data randomly
```

Bug 4:

```
forgetting model.eval() for evaluation
```

Mini-Check: BatchNorm Eval Bug

A student evaluates the same checkpoint twice.

Validation accuracy changes noticeably when:

```
batch size changes  
batch order changes  
validation loader is shuffled
```

Which evaluation step should you check first?

Answer

Check whether the model was switched to evaluation mode:

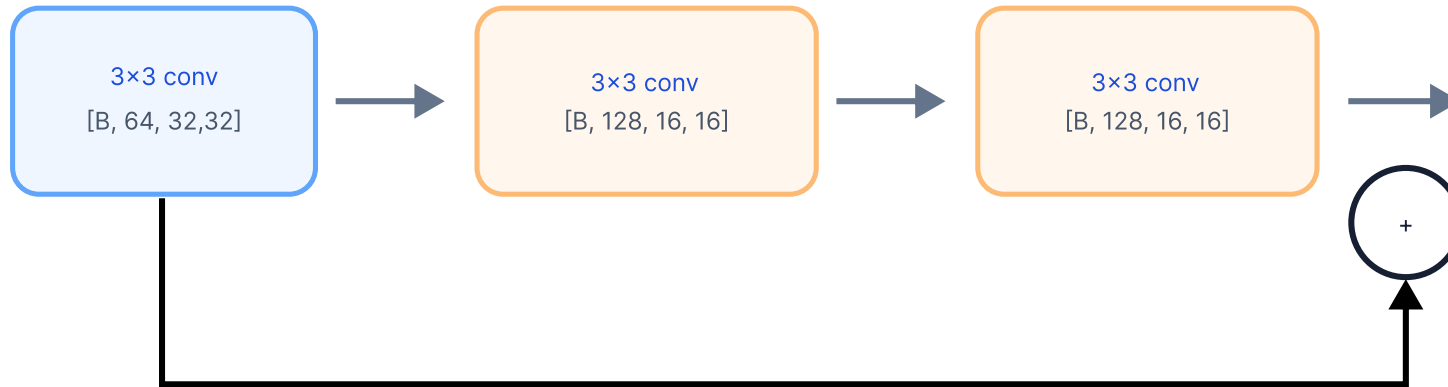
```
model.eval()
```

If BatchNorm stays in training mode, it uses current mini-batch statistics.

That can make validation depend on batch composition.

Mini-Check: Residual Shape Bug

Can we add these two tensors directly?



Can we compute $y = F(x) + x$ directly here?

What needs to change?

Answer

No.

Residual addition needs exactly matching shapes.

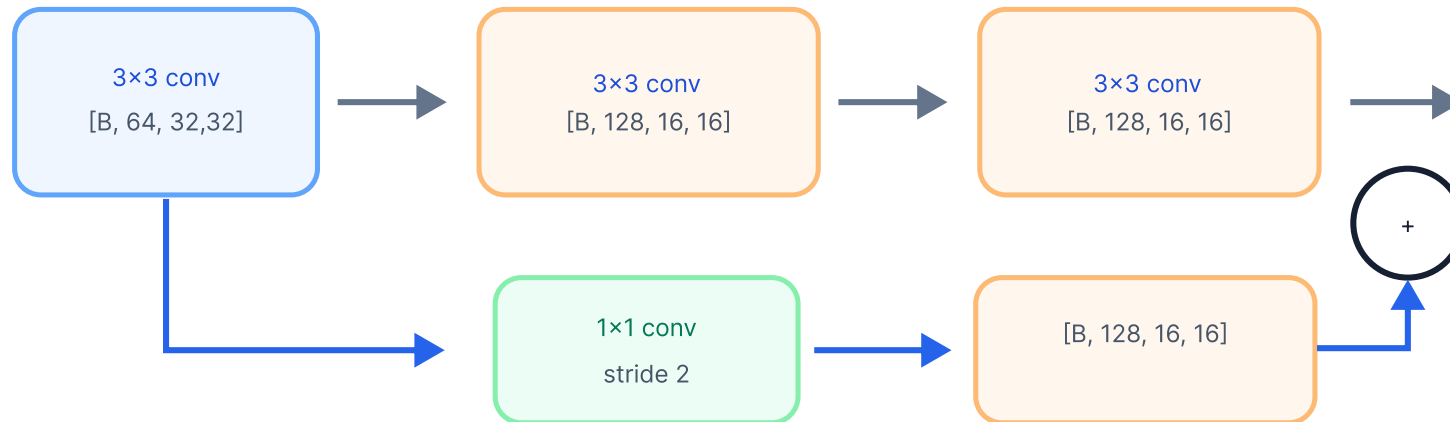
Here both channel count and spatial size differ:

```
x:      [B, 64, 32, 32]  
F(x): [B, 128, 16, 16]
```

Use a projection shortcut, often a `1 x 1` convolution with `stride=2`.

Answer

Can we add these two tensors directly?



Use a projection shortcut, often a `1 x 1` convolution with `stride=2`.

Seminar Bridge

In Seminar 06, you will:

1. train a baseline CNN on CIFAR-10
2. add BatchNorm and compare
3. add Alumentations and compare
4. create a comparison table
5. optionally implement a tiny residual block

Main question:

which change improves training, generalization, or both?

Takeaways

For deeper CNN training:

- BatchNorm stabilizes intermediate activations
- BatchNorm has different train/eval behavior
- augmentation improves data diversity
- validation should stay clean
- residual connections help gradients and information flow

Next lecture:

using pretrained CNNs instead of training from scratch